

UNIDAD 4 · SEMANAS 5-6

Procesos y memoria

📅 Semanas 5-6 · fechas por confirmar </> Laboratorio TD3 (resuelto + desafío IA)

~ 7 animaciones

MANUAL M4 · TEORÍA DE LA UNIDAD

Procesos y memoria

Lectura base de la unidad 4 (≈ 2 h, dos semanas). Cada sección termina con un enlace **Practica esto** →. Las salidas son reales: programas del TD3 ejecutados en el equipo del curso (Ubuntu 26.04, Linux 7.0, x86-64).

CONTENIDO

- | | | | |
|----|---|----|---|
| §1 | El proceso y su espacio de direcciones | §4 | <code>fork</code> , copia en escritura y búferes |
| §2 | Memoria virtual: páginas, fallos y reemplazo | §5 | <code>exec</code> , <code>exit</code> y <code>wait</code> |
| §3 | El proceso para el núcleo: PCB, estados y planificación | §6 | El primer proceso, sesiones y servicios |

§1 El proceso y su espacio de direcciones

Un **proceso** es un programa en ejecución junto con su contexto: memoria, registros, descriptores abiertos, credenciales. Cada proceso tiene un **espacio de direcciones privado**: las direcciones que usa son *virtuales*, y la misma dirección en dos procesos apunta a memoria física distinta. En x86-64 los punteros tienen 64 bits pero se usan 48: 256 TiB, la mitad baja para el proceso y la alta para el núcleo.

El espacio se organiza en **zonas**: el *código* (texto, de solo lectura y ejecutable), los *datos* (globales inicializadas) y el *bss* (globales sin inicializar), el *heap* (`malloc` pequeños), las regiones *mmap* (bibliotecas y `malloc` grandes) y la *pila* (variables locales, que crece hacia abajo). El programa `zones` del TD3 imprime una dirección de cada zona y la región de `/proc/self/maps` que la contiene:

Salida real

```
$ ./zones
zone                address    perm  region
code   (main)       0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)  0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)       0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)          0x7fff6c1e68b4  rw-p  [stack]
```

Si lo ejecutas varias veces, las direcciones cambian: el núcleo sortea la base de cada región (**ASLR**, *address space layout randomization*) para que un atacante no pueda predecir dónde está el código o la pila. glibc sirve los `malloc` de 128 KiB o más con un `mmap` propio, fuera del heap, que devuelve al sistema en cuanto se libera.

Practica esto → TD3 · 5 (zones)

§2 Memoria virtual: páginas, fallos y reemplazo

La memoria se divide en **páginas** (4 KiB en x86-64; también páginas enormes de 2 MiB y 1 GiB) y la física en **marcos** del mismo tamaño. Una **tabla de páginas** por proceso dice en qué marco está cada página y con qué permisos (lectura, escritura, ejecución, usuario). En x86-64 la tabla tiene 4 niveles: una dirección virtual se parte en cuatro índices de 9 bits y un desplazamiento de 12. La traducción la hace el hardware —la **MMU**— en cada acceso, y la **TLB** guarda las traducciones recientes para no recorrer la tabla cada vez.

Si la entrada de la tabla no está presente, la MMU produce un **fallo de página** y salta al núcleo. Si la dirección pertenece a una región válida, el núcleo asigna un marco, lo pone en cero (o lo lee del disco) y repite la instrucción: es la **asignación perezosa**. Si no pertenece a ninguna región, o el acceso viola los permisos (escribir en código, ejecutar en la pila), envía `SIGSEGV`. Medido con `lazy`:

Salida real

```
$ ./lazy 1024
start                VmRSS =    1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc         VmRSS =    1952 KiB    page faults so far = 8
after touching half of the pages VmRSS =  526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each) page faults: 131073    1968 ns per fault
after free           VmRSS =    2012 KiB    page faults so far = 131081
```

Linux es además *optimista* (*overcommit*): promete más memoria de la que tiene, porque casi nadie usa todo lo que reserva. Si de verdad se agota, el **OOM killer** termina procesos. Cuando no quedan marcos libres, el núcleo elige una página **víctima** y la lleva al disco (*swap*) o la comprime en RAM (*zswap*, *zram*). El algoritmo ideal (OPT) sacaría la página que se usará más tarde; los reales aproximan LRU con el bit de acceso que enciende la MMU (algoritmo del reloj; en Linux, listas activa/inactiva y MGLRU). Con FIFO puede ocurrir la **anomalía de Belady**: más marcos, más fallos. Si el conjunto de páginas en uso no cabe en memoria, el sistema pasa más tiempo paginando que ejecutando: **hiperpaginación**.

Practica esto → A4.1 memoria virtual · A4.5 reemplazo de páginas

§3 El proceso para el núcleo: PCB, estados y planificación

El núcleo describe cada proceso con un **bloque de control** (PCB; en Linux, `struct task_struct`): PID y PPID, dueño y permisos, estado, registros guardados, prioridad, tiempos de CPU, tabla de descriptores, límites (`getrlimit`), señales y tabla de páginas. Se puede consultar en `/proc/PID/`, que es lo que leen `ps` y `top`.

Un proceso pasa por varios **estados**: *listo* (espera la CPU) y *ejecutando* —ambos `R` en `ps`—, *bloqueado* esperando E/S (`S`, o `D` si no se puede interrumpir), *detenido* por `SIGSTOP` / `SIGTSTP` (`T`) y *zombi* (`Z`) entre su final y el `wait` de su padre.

Pasar la CPU de un proceso a otro es un **cambio de contexto**: guardar los registros de uno, cargar los del otro y cambiar de tabla de páginas. El **planificador** decide a quién le toca. Los algoritmos clásicos: FCFS (orden de llegada; sufre el efecto convoy), SJF y SRTF (el más corto primero; minimizan la espera media pero exigen estimar duraciones), Round Robin (turnos de un *quantum*; buena respuesta, más cambios de contexto) y prioridades (con riesgo de inanición, que se corrige con envejecimiento). Linux usa desde la versión 6.6 (2023) **EEVDF**, que reparte la CPU por tiempo virtual ponderado por la prioridad (`nice`) y atiende primero el plazo más cercano.

Practica esto → A4.3 estados · A4.4 planificación

§4 `fork`, copia en escritura y búferes

En Unix un proceso nuevo nace como **clon** de otro: `fork()` devuelve 0 en el hijo, el PID del hijo en el padre y `-1` si falla. El hijo recibe una copia de todo: memoria, descriptores (que apuntan a las mismas entradas de la tabla de archivos abiertos, unidad 3), manejadores y máscara de señales —pero no las señales pendientes—. Copiar toda la memoria sería carísimo, así que Linux usa **copia en escritura**: copia solo las tablas de páginas, marca todo como de solo lectura y copia cada página recién cuando alguno de los dos escribe en ella:

Salida real

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per copy
parent: the child exited with status 0; my copy still says 'x'
```

La copia incluye también lo que la biblioteca de C tenga pendiente. `printf` no llama a `write`: acumula en un **búfer** que se vacía por línea (stdout hacia una terminal), cuando se llena (stdout hacia una tubería o un archivo), con `fflush` o al salir con `exit`; stderr no tiene búfer y `_exit` no vacía nada. Si al hacer `fork` quedaba texto en el búfer, padre e hijo lo escriben: sale dos veces.

Salida real

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Practica esto → A4.2 copia en escritura · A4.6 búferes · TD3 · 2 y 3

§5 **exec, exit y wait**

`execve(path, argv, envp)` reemplaza el programa del proceso: carga código y datos nuevos, crea un heap y una pila nuevos y salta al punto de entrada. Conserva el PID, el PPID y los descriptores abiertos (salvo los marcados `O_CLOEXEC`); con el bit SUID, el UID efectivo pasa a ser el del dueño del archivo. Si funciona, no vuelve. Las variantes de biblioteca (`execl`, `execv`, `execvp` ...) cambian cómo se pasan los argumentos y si se busca en el `PATH`. Entre `fork` y `exec` el hijo puede reorganizar sus descriptores: así el shell implementa `>`, `<` y las tuberías (`launch` del TD3 · 4, y la unidad 5).

Un proceso termina con `exit(status)` —que llama a las funciones de `atexit`, vacía los búferes de stdio y llama a la llamada al sistema `_exit`— o por una señal. El núcleo cierra sus archivos y libera su memoria, entrega sus hijos vivos a `init` o al *subreaper* más cercano y avisa al padre con `SIGCHLD`. El PCB queda como **zombi** hasta que el padre llama a `wait` o `waitpid`, que devuelve el estado empaquetado; las macros lo decodifican: `WIFEXITED` / `WEXITSTATUS`, `WIFSIGNALED` / `WTERMSIG`, `WIFSTOPPED` y `WIFCONTINUED` (con las opciones `WUNTRACED` y `WCONTINUED`; `WNOHANG` no bloquea).

Salida real

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
   grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

Salida real

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

Practica esto → A4.7 fork, exec y wait · TD3 · 4

§6 El primer proceso, sesiones y servicios

Si todo proceso nace de un `fork`, alguien tiene que ser el primero. El **proceso 0** lo construye el núcleo al arrancar (uno por CPU, en modo núcleo): inicializa la memoria virtual, crea los hilos del núcleo (`kthreadd`, PID 2) y el **proceso 1**, que pasa a modo usuario y ejecuta `/sbin/init` —hoy **systemd** en casi todas las distribuciones—, arranca los servicios y adopta a los huérfanos. En el equipo del curso, `ps -p 1 -o comm=` responde `systemd` (TD0).

Una **sesión** agrupa procesos, típicamente los de una terminal, con un líder; si el líder termina, los demás reciben `SIGHUP`. `setsid()` crea una sesión nueva sin terminal de control. Un **demonio** clásico se desprendía de la terminal con `fork`, el padre salía y el hijo llamaba a `setsid`. Hoy, en Linux, los servicios los lanza `systemd`: el programa corre en primer plano y `systemd` se ocupa de la sesión, los registros (`journalctl`), los reinicios y los límites de recursos (`cgroups`, unidad 9).